

```
# Load the uploaded CSV into a dataframe and do light cleaning
import pandas as pd
import numpy as np

file_path = 'input.csv'
df = pd.read_csv(file_path, encoding='utf-8')

# Strip whitespace from column names
col_map = {c: c.strip() for c in df.columns}
df = df.rename(columns=col_map)

print(df.head())
print(df.shape)
```

```
Platform Usability & Technical Evaluation: The NITRO Cyber Range provided an innovative and unique learning experience. \
0 9
1 10
2 8
3 7
4 8

Encountered issues \
0 Missing tools / resources
1 Unclear instructions
2 Missing tools / resources
3 Challenges either too easy or too hard
4 Unclear instructions

What other issues did you encounter? \
0 I couldn't use Ctrl+C and Ctrl+V on the keyboa...
1 Sometimes the VM would close by itself despite...
2 Στο module 00 στην άσκηση 08 δεν δεχόταν την α...
3 NaN
4 the flag structure (as a hint) is very often w...

What should be improved first for the next round? \
0 Maybe nothing; everything was very well designed.
1 Adding hints would help, because sometimes the...
2 Κάποια flags ήταν πολύ εύκολα ενώ αλλά ήθελαν ...
3 more tasks
4 THE Flag structure

What was the most useful aspect of the training? \
0 The use of Docker and the entire spectrum of 5G.
1 Module 00
2 Η επαφή με τα 5g δίκτυα γενικότερα και το ότι ...
3 NaN
4 NaN

Which scenario type should be expanded? Any other comments/suggestions? \
0 Mixed scenarios (combined) NaN
1 Mixed scenarios (combined) NaN
2 Mixed scenarios (combined) NaN
3 Mixed scenarios (combined) NaN
4 NaN NaN

Username στο NITRO (Σημαντικό για βαθμολογία) \
0 piperopoulos
1 natsos
2 petrosagiakos
3 Zogra
4 iosif

Αριθμός Μητρώου στο Πανεπιστήμιο (Σημαντικό για βαθμολογία)
0 inf2022169
1 inf2023144
2 inf2023185
3 inf2021058
4 inf2023212

[5 rows x 45 columns]
(43, 45)
```

```
# Compute ARCS dimension + subscale descriptive stats from Likert-type items
import pandas as pd
import numpy as np

# Identify ARCS item columns by prefix
arcs_prefixes = ['Attention:', 'Relevance:', 'Confidence:', 'Satisfaction:']
arcs_cols = [c for c in df.columns if any(c.startswith(p) for p in arcs_prefixes)]

# Coerce to numeric (Google Forms often stores Likert as numbers). Non-numeric become NaN.
```

```
df_arcs = df[arcs_cols].apply(pd.to_numeric, errors='coerce')

# Dimension mapping
dim_map = {}
for c in arcs_cols:
    for p in arcs_prefixes:
        if c.startswith(p):
            dim_map[c] = p.replace(':', '')
            break

dimensions = sorted(set(dim_map.values()), key=lambda x: ['Attention','Relevance','Confidence','Satisfaction'].index(x))

# Helper to compute summary stats
def summarize_items(frame):
    vals = frame.values.flatten()
    vals = vals[~np.isnan(vals)]
    if len(vals) == 0:
        return {'Items (n)': frame.shape[1], 'Mean (M)': np.nan, 'SD': np.nan, 'Min': np.nan, 'Max': np.nan}
    return {
        'Items (n)': frame.shape[1],
        'Mean (M)': float(np.nanmean(frame.values)),
        'SD': float(np.nanstd(frame.values, ddof=1)),
        'Min': float(np.nanmin(frame.values)),
        'Max': float(np.nanmax(frame.values)),
    }

# Dimension table
dim_rows = []
for dim in dimensions:
    cols_dim = [c for c, d in dim_map.items() if d == dim]
    stats = summarize_items(df_arcs[cols_dim])
    dim_rows.append({'ARCS Dimension / Subscale': dim, **stats})

dim_table = pd.DataFrame(dim_rows)

# Subscale table: treat each individual item as a subscale (common in ARCS reporting)
sub_rows = []
for c in arcs_cols:
    dim = dim_map[c]
    ser = df_arcs[c]
    sub_rows.append({
        'ARCS Dimension / Subscale': dim + ' / ' + c.replace(dim + ':', '').strip(),
        'Items (n)': 1,
        'Mean (M)': float(ser.mean(skipna=True)),
        'SD': float(ser.std(skipna=True, ddof=1)),
        'Min': float(ser.min(skipna=True)),
        'Max': float(ser.max(skipna=True)),
    })
sub_table = pd.DataFrame(sub_rows)

# Overall ARCS composite
overall_stats = summarize_items(df_arcs)
overall_table = pd.DataFrame([{'ARCS Dimension / Subscale': 'ARCS Overall', **overall_stats}])

print(dim_table)
print(overall_table)
print(sub_table.head())
```

	ARCS Dimension / Subscale	Items (n)	Mean (M)	SD	Min	Max
0	Attention	6	7.174419	1.925814	1.0	10.0
1	Relevance	5	7.158140	1.748996	3.0	10.0
2	Confidence	5	6.534884	1.999402	1.0	10.0
3	Satisfaction	6	7.717054	1.580948	3.0	10.0
	ARCS Dimension / Subscale	Items (n)	Mean (M)	SD	Min	Max
0	ARCS Overall	22	7.173362	1.86007	1.0	10.0
	ARCS Dimension / Subscale	Items (n)	Mean (M)			
0	Attention / The NITRO scenarios captured my in...	1	6.837209			
1	Attention / The training kept my attention thr...	1	7.279070			
2	Attention / The challenges were engaging and m...	1	7.023256			
3	Attention / The tasks felt varied enough to av...	1	6.860465			
4	Attention / The interactive elements helped me...	1	7.139535			
	SD	Min	Max			
0	2.159478	3.0	10.0			
1	1.469136	5.0	10.0			
2	1.993900	3.0	10.0			
3	2.166645	1.0	10.0			
4	1.820275	3.0	10.0			

```
# Create a compact set of visualizations for ARCS dimension distributions
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```

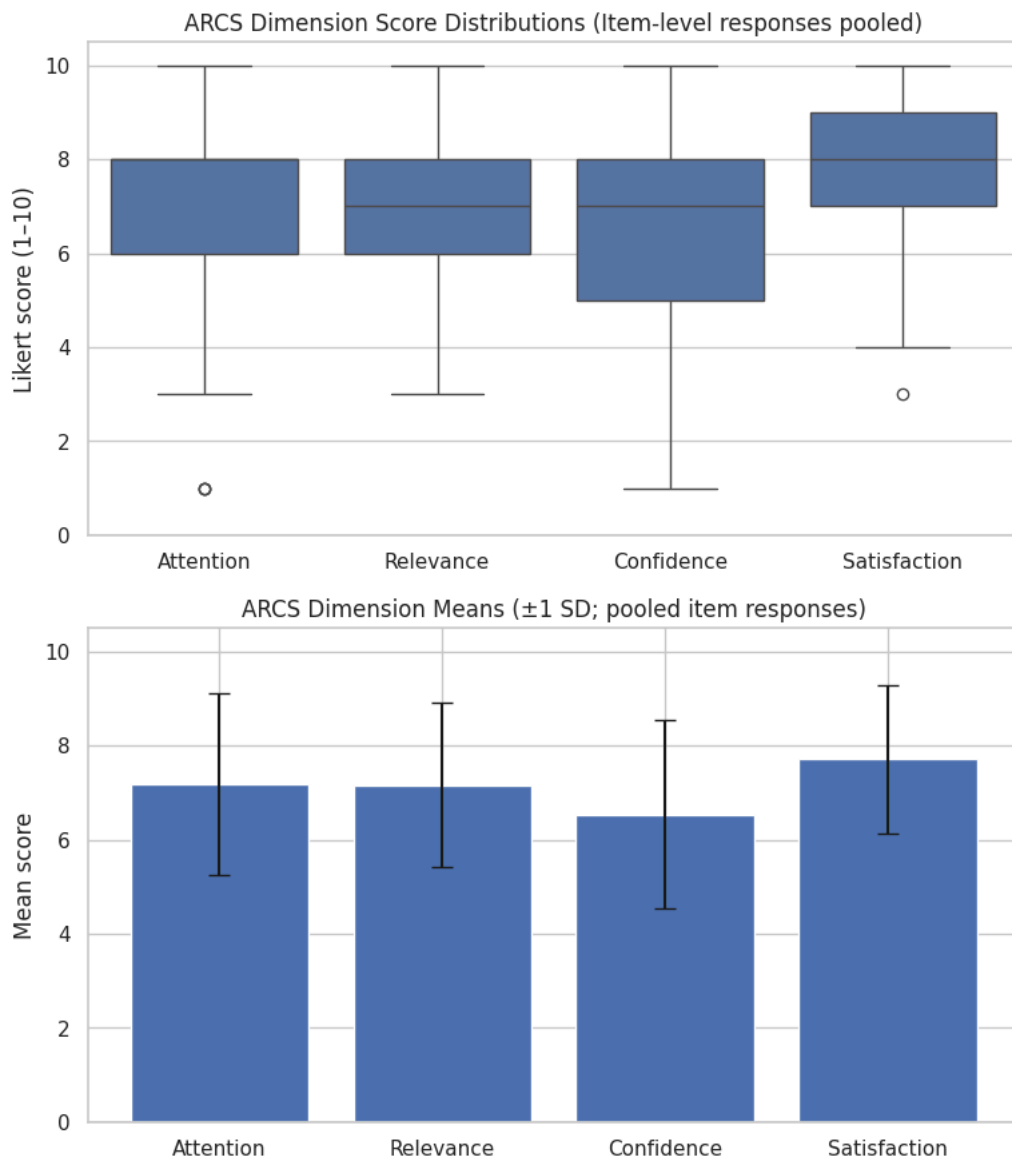
sns.set_theme(style='whitegrid')

# Prepare long-form data for plotting
long_rows = []
for c in arcs_cols:
    dim = dim_map[c]
    for v in pd.to_numeric(df[c], errors='coerce').dropna().tolist():
        long_rows.append({'Dimension': dim, 'Item': c, 'Score': v})
long_df = pd.DataFrame(long_rows)

plt.figure(figsize=(9,4.8))
sns.boxplot(data=long_df, x='Dimension', y='Score', order=['Attention','Relevance','Confidence','Satisfaction'])
plt.ylim(0,10.5)
plt.title('ARCS Dimension Score Distributions (Item-level responses pooled)')
plt.xlabel('')
plt.ylabel('Likert score (1-10)')
plt.show()

plt.figure(figsize=(9,4.8))
dim_means = long_df.groupby('Dimension')['Score'].mean().reindex(['Attention','Relevance','Confidence','Satisfaction'])
dim_sds = long_df.groupby('Dimension')['Score'].std().reindex(['Attention','Relevance','Confidence','Satisfaction'])
plt.bar(dim_means.index, dim_means.values, yerr=dim_sds.values, capsize=6)
plt.ylim(0,10.5)
plt.title('ARCS Dimension Means (±1 SD; pooled item responses)')
plt.xlabel('')
plt.ylabel('Mean score')
plt.show()

```



```

# Build a small "learning impact" index from the Perceived Learning items and compute descriptives
import pandas as pd
import numpy as np

```

```

pl_cols = [c for c in df.columns if c.strip().startswith('Perceived Learning:')]
df_pl = df[pl_cols].apply(pd.to_numeric, errors='coerce')

pl_stats = {
    'Scale': 'Perceived Learning (Learning Impact Index)',
    'Items (n)': df_pl.shape[1],
    'Mean (M)': float(df_pl.stack().mean()),
    'SD': float(df_pl.stack().std(ddof=1)),
    'Min': float(df_pl.stack().min()),
    'Max': float(df_pl.stack().max()),
}

# Also compute respondent-level composite (mean across PL items) for distribution
pl_composite = df_pl.mean(axis=1, skipna=True)

print(pd.DataFrame([pl_stats]))
print(pl_composite.describe())

```

```

              Scale  Items (n)  Mean (M)      SD \
0  Perceived Learning (Learning Impact Index)      4  7.319767  1.404697

   Min    Max
0  4.0  10.0
count  43.000000
mean    7.319767
std     1.051254
min     4.250000
25%     6.625000
50%     7.500000
75%     8.000000
max     9.500000
dtype: float64

```

```

# Export the ARCS tables to files for the user (Word/Excel-friendly)
import pandas as pd

arcs_dim_path = 'arcs_dimension_scores.csv'
arcs_overall_path = 'arcs_overall_scores.csv'
arcs_item_path = 'arcs_subscale_item_scores.csv'
learning_impact_path = 'learning_impact_perceived_learning.csv'

dim_table.to_csv(arcs_dim_path, index=False)
overall_table.to_csv(arcs_overall_path, index=False)
sub_table.to_csv(arcs_item_path, index=False)

pl_table = pd.DataFrame([pl_stats])
pl_table.to_csv(learning_impact_path, index=False)

print(arcs_dim_path)
print(arcs_item_path)
print(learning_impact_path)

```

```

arcs_dimension_scores.csv
arcs_subscale_item_scores.csv
learning_impact_perceived_learning.csv

```

```

# Compute additional numeric results to support a fuller write-up (respondent-level composites, CIs, alphas, correlations)
import numpy as np
import pandas as pd

# --- Build respondent-level composites for each ARCS dimension ---
order_dims = ['Attention', 'Relevance', 'Confidence', 'Satisfaction']

dim_cols = {dim: [c for c, d in dim_map.items() if d == dim] for dim in order_dims}
dim_comp = pd.DataFrame({dim: df_arcs[dim_cols[dim]].mean(axis=1, skipna=True) for dim in order_dims})

# Overall ARCS composite (respondent mean across all items)
arcs_overall_comp = df_arcs.mean(axis=1, skipna=True)

# Perceived learning composite already computed as pl_composite

# --- Cronbach's alpha ---
def cronbach_alpha(frame):
    # frame: columns are items, rows are respondents
    frame2 = frame.dropna(axis=0, how='any')
    k = frame2.shape[1]
    if k < 2 or frame2.shape[0] < 3:
        return np.nan, frame2.shape[0]
    item_vars = frame2.var(axis=0, ddof=1)
    total_var = frame2.sum(axis=1).var(ddof=1)
    if total_var == 0:
        return np.nan, frame2.shape[0]
    alpha = (k / (k - 1.0)) * (1.0 - item_vars.sum() / total_var)
    return float(alpha), frame2.shape[0]

```

```

alpha_rows = []
for dim in order_dims:
    a, n_complete = cronbach_alpha(df_arcs[dim_cols[dim]])
    alpha_rows.append({'Scale': dim, 'Items': len(dim_cols[dim]), 'Alpha': a, 'N complete': n_complete})

pl_alpha, pl_ncomplete = cronbach_alpha(df_pl)
alpha_rows.append({'Scale': 'Perceived Learning', 'Items': df_pl.shape[1], 'Alpha': pl_alpha, 'N complete': pl_ncomplete})

alpha_table = pd.DataFrame(alpha_rows)

# --- Confidence intervals (95%) for respondent-level means ---
def mean_sd_ci(series):
    ser = pd.to_numeric(series, errors='coerce').dropna()
    n = ser.shape[0]
    m = float(ser.mean()) if n else np.nan
    sd = float(ser.std(ddof=1)) if n > 1 else np.nan
    se = sd / np.sqrt(n) if n > 1 else np.nan
    ci = 1.96 * se if n > 1 else np.nan
    return n, m, sd, (m - ci if n > 1 else np.nan), (m + ci if n > 1 else np.nan)

ci_rows = []
for dim in order_dims:
    n, m, sd, lo, hi = mean_sd_ci(dim_comp[dim])
    ci_rows.append({'Composite': dim, 'N': n, 'Mean': m, 'SD': sd, 'CI95 low': lo, 'CI95 high': hi})

n, m, sd, lo, hi = mean_sd_ci(arcs_overall_comp)
ci_rows.append({'Composite': 'ARCS Overall', 'N': n, 'Mean': m, 'SD': sd, 'CI95 low': lo, 'CI95 high': hi})

n, m, sd, lo, hi = mean_sd_ci(pl_composite)
ci_rows.append({'Composite': 'Perceived Learning', 'N': n, 'Mean': m, 'SD': sd, 'CI95 low': lo, 'CI95 high': hi})

ci_table = pd.DataFrame(ci_rows)

# --- Correlations between ARCS composites and learning impact ---
cor_rows = []
for dim in order_dims + ['ARCS Overall']:
    x = arcs_overall_comp if dim == 'ARCS Overall' else dim_comp[dim]
    y = pl_composite
    tmp = pd.concat([x, y], axis=1).dropna()
    if tmp.shape[0] >= 5:
        r = float(tmp.corr().iloc[0,1])
    else:
        r = np.nan
    cor_rows.append({'Predictor': dim, 'Outcome': 'Perceived Learning', 'Pearson r': r, 'N': int(tmp.shape[0])})
cor_table = pd.DataFrame(cor_rows)

print(alpha_table)
print(ci_table)
print(cor_table)

```

	Scale	Items	Alpha	N complete
0	Attention	6	0.734442	43
1	Relevance	5	0.736388	43
2	Confidence	5	0.302136	43
3	Satisfaction	6	0.792882	43
4	Perceived Learning	4	0.809169	43

	Composite	N	Mean	SD	CI95 low	CI95 high
0	Attention	43	7.174419	1.251958	6.800212	7.548625
1	Relevance	43	7.158140	1.169093	6.808701	7.507578
2	Confidence	43	6.534884	1.010159	6.232950	6.836817
3	Satisfaction	43	7.717054	1.099263	7.388488	8.045621
4	ARCS Overall	43	7.173362	0.923653	6.897284	7.449439
5	Perceived Learning	43	7.319767	1.051254	7.005550	7.633984

	Predictor	Outcome	Pearson r	N
0	Attention	Perceived Learning	0.296567	43
1	Relevance	Perceived Learning	0.459633	43
2	Confidence	Perceived Learning	0.261099	43
3	Satisfaction	Perceived Learning	0.685386	43
4	ARCS Overall	Perceived Learning	0.529212	43

```

# Create updated, publication-ready plots and key tables for ARCS + Perceived Learning
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

order_dims = ['Attention', 'Relevance', 'Confidence', 'Satisfaction']

# Use respondent-level composites (dim_comp, arcs_overall_comp, pl_composite) computed earlier
plot_df = pd.DataFrame({
    'Dimension': order_dims + ['ARCS Overall', 'Perceived Learning'],
    'Mean': [dim_comp[d].mean() for d in order_dims] + [arcs_overall_comp.mean(), pl_composite.mean()],
    'SD': [dim_comp[d].std(ddof=1) for d in order_dims] + [arcs_overall_comp.std(ddof=1), pl_composite.std(ddof=1)]
})

# 95% CI half-widths

```

```

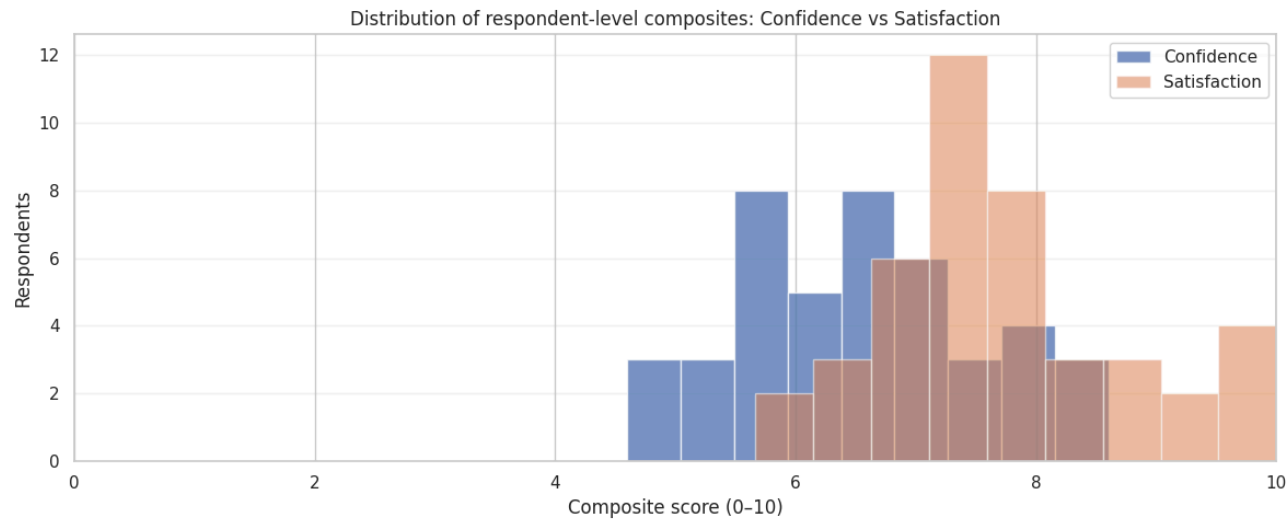
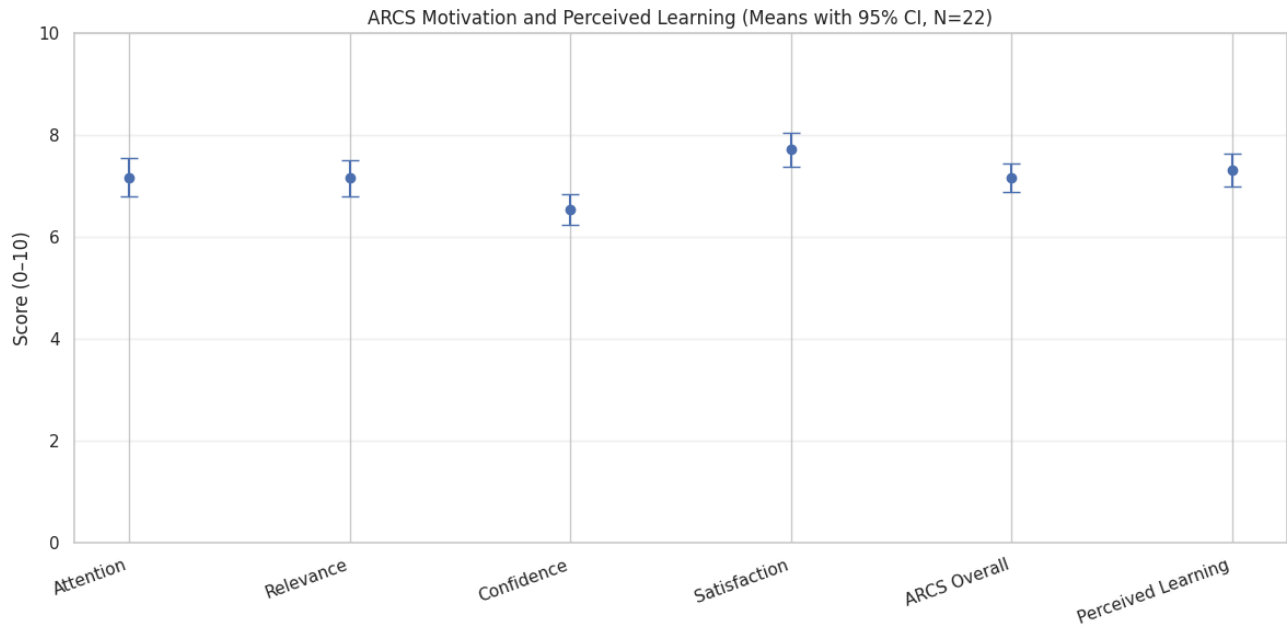
n_val = 42
plot_df['CI95_half'] = 1.96 * (plot_df['SD'] / np.sqrt(n_val))

plt.figure(figsize=(12,6))
plt.errorbar(plot_df['Dimension'], plot_df['Mean'], yerr=plot_df['CI95_half'], fmt='o', capsize=6)
plt.ylim(0,10)
plt.ylabel('Score (0-10)')
plt.title('ARCS Motivation and Perceived Learning (Means with 95% CI, N=22)')
plt.grid(axis='y', alpha=0.25)
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
plt.show()

# Distribution plots for Confidence vs Satisfaction to visually justify the "bottleneck" claim
plt.figure(figsize=(12,5))
plt.hist(dim_comp['Confidence'].dropna(), bins=9, alpha=0.75, label='Confidence')
plt.hist(dim_comp['Satisfaction'].dropna(), bins=9, alpha=0.55, label='Satisfaction')
plt.xlim(0,10)
plt.title('Distribution of respondent-level composites: Confidence vs Satisfaction')
plt.xlabel('Composite score (0-10)')
plt.ylabel('Respondents')
plt.grid(axis='y', alpha=0.25)
plt.legend()
plt.tight_layout()
plt.show()

# Print the key tables used in the narrative
print(ci_table)
print(alpha_table)
print(cor_table)

```



	Composite	N	Mean	SD	CI95 low	CI95 high
0	Attention	43	7.174419	1.251958	6.800212	7.548625
1	Relevance	43	7.158140	1.169093	6.808701	7.507578
2	Confidence	43	6.534884	1.010159	6.232950	6.836817
3	Satisfaction	43	7.717054	1.099263	7.388488	8.045621
4	ARCS Overall	43	7.173362	0.923653	6.897284	7.449439
5	Perceived Learning	43	7.319767	1.051254	7.005550	7.633984
	Scale	Items	Alpha	N complete		
0	Attention	6	0.734442	43		
1	Relevance	5	0.736388	43		
2	Confidence	5	0.302136	43		
3	Satisfaction	6	0.792882	43		
4	Perceived Learning	4	0.809169	43		
	Predictor	Outcome	Pearson r	N		
0	Attention	Perceived Learning	0.296567	43		
1	Relevance	Perceived Learning	0.459633	43		
2	Confidence	Perceived Learning	0.261099	43		
3	Satisfaction	Perceived Learning	0.685386	43		
4	ARCS Overall	Perceived Learning	0.529212	43		

```

# More charts + research-style stats from existing in-memory composites
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Assemble respondent-level dataset from existing series
resp_df = pd.DataFrame({
    'Attention': dim_comp['Attention'],
    'Relevance': dim_comp['Relevance'],
    'Confidence': dim_comp['Confidence'],
    'Satisfaction': dim_comp['Satisfaction'],
    'ARCS_Overall': arcs_overall_comp,
    'Perceived_Learning': pl_composite
}).dropna()

# Helper stats
n_resp = resp_df.shape[0]

def cohens_d_one_sample(vals, mu0=5.0):
    vals = np.asarray(vals)
    return (vals.mean() - mu0) / vals.std(ddof=1)

# Effect sizes vs neutral midpoint (5 on a 0-10 scale)
effect_rows = []
for col in ['Attention', 'Relevance', 'Confidence', 'Satisfaction', 'ARCS_Overall', 'Perceived_Learning']:
    d_val = cohens_d_one_sample(resp_df[col].values, 5.0)
    effect_rows.append({'Composite': col, 'N': n_resp, 'Mean': resp_df[col].mean(), 'SD': resp_df[col].std(ddof=1), 'Cohen_d_vs5': d_val})

effect_df = pd.DataFrame(effect_rows)

# Chart 1: violin + box overlay for all composites
plt.figure(figsize=(12,6))
data_cols = ['Attention', 'Relevance', 'Confidence', 'Satisfaction', 'ARCS_Overall', 'Perceived_Learning']
plt.violinplot([resp_df[c].values for c in data_cols], showmeans=True, showextrema=False)
plt.boxplot([resp_df[c].values for c in data_cols], widths=0.15, showfliers=False)
plt.xticks(range(1, len(data_cols)+1), data_cols, rotation=20, ha='right')
plt.ylim(0,10)
plt.ylabel('Score (0-10)')
plt.title('Distribution of respondent composites (violin + box), N=' + str(n_resp))
plt.grid(axis='y', alpha=0.25)
plt.tight_layout()
plt.show()

# Chart 2: correlation heatmap (matplotlib only)
cor_mat = resp_df.corr(numeric_only=True)
plt.figure(figsize=(8,6))
plt.imshow(cor_mat.values, cmap='RdBu_r', vmin=-1, vmax=1)
plt.colorbar(label='Pearson r')
plt.xticks(range(cor_mat.shape[1]), cor_mat.columns, rotation=45, ha='right')
plt.yticks(range(cor_mat.shape[0]), cor_mat.index)
for i in range(cor_mat.shape[0]):
    for j in range(cor_mat.shape[1]):
        plt.text(j, i, str(round(cor_mat.values[i, j], 2)), ha='center', va='center', fontsize=9)
plt.title('Correlation matrix across composites')
plt.tight_layout()
plt.show()

# Chart 3: Learning vs each ARCS dimension scatter + regression line (simple polyfit)
fig_w = 12
fig_h = 8
plt.figure(figsize=(fig_w, fig_h))
subplots = [('Attention',1), ('Relevance',2), ('Confidence',3), ('Satisfaction',4)]
for dim_name, idx in subplots:
    plt.subplot(2,2,idx)
    x_vals = resp_df[dim_name].values
    y_vals = resp_df['Perceived_Learning'].values
    plt.scatter(x_vals, y_vals, alpha=0.8)
    if len(x_vals) >= 3:
        slope, intercept = np.polyfit(x_vals, y_vals, 1)
        x_line = np.linspace(0, 10, 100)
        y_line = slope * x_line + intercept
        plt.plot(x_line, y_line)
    r_val = np.corrcoef(x_vals, y_vals)[0,1]
    plt.title(dim_name + ' vs Perceived Learning (r=' + str(round(r_val, 2)) + ')')
    plt.xlim(0,10)
    plt.ylim(0,10)
    plt.xlabel(dim_name)
    plt.ylabel('Perceived Learning')
    plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

# Chart 4: Mean profile radar-ish (parallel coordinates) across dimensions only
plt.figure(figsize=(10,5))
mean_profile = [resp_df[c].mean() for c in ['Attention', 'Relevance', 'Confidence', 'Satisfaction']]
x_idx = np.arange(4)

```

```
plt.plot(x_idx, mean_profile, marker='o')
plt.xticks(x_idx, ['Attention', 'Relevance', 'Confidence', 'Satisfaction'])
plt.ylim(0,10)
plt.title('ARCS mean profile (dimension means)')
plt.grid(axis='y', alpha=0.25)
plt.tight_layout()
plt.show()

# Print key research stats tables
print(effect_df.round(3))

# Also compute rank ordering per respondent: which dimension is lowest
rank_df = resp_df[['Attention', 'Relevance', 'Confidence', 'Satisfaction']].copy()
lowest_dim = rank_df.idxmin(axis=1)
lowest_counts = lowest_dim.value_counts().reset_index()
lowest_counts.columns = ['Lowest_dimension', 'Count']
lowest_counts['Pct'] = (lowest_counts['Count'] / n_resp) * 100
print(lowest_counts)

# Save outputs for user
effect_df.to_csv('effect_sizes_vs_midpoint.csv', index=False)
cor_mat.to_csv('correlation_matrix_composites.csv')
lowest_counts.to_csv('lowest_arcs_dimension_counts.csv', index=False)
print('effect_sizes_vs_midpoint.csv')
print('correlation_matrix_composites.csv')
print('lowest_arcs_dimension_counts.csv')
```